# Task 1: Organisational and System analysis

## Overview of selected organisation:

The Warehouse Group (TWG) is New Zealand's largest listed retailer, with almost 12,000 employees across its large distribution network of stores and the support centre. From small beginnings in a single Auckland warehouse in 1982, TWG has grown to become New Zealand's largest retail group, and includes some of New Zealand's best known brands – The Warehouse, Warehouse Stationery, Noel Leeming and Torpedo7 – selling general merchandise, electronics, office products as well as outdoor and sporting equipment.

TWG is competing with many other retail shops in the market place where efficiency, inventory accuracy and customer satisfaction are key factors to their success. Integrated Information Systems play a vital role in TWG in this era where the operations of several departments within any organization need to be coordinated in order for the company to run smoothly. The Retail Management System (RMS) is a business-critical system within the organisation.

The RMS from Retail Journal centralises TWG's various channels of data enabling the business to accurately monitor and manage stock levels across all retail platforms. Without a central management system to handle customer transactions across in-store and online platforms, TWG would face problems with out-of-date stock levels, slow processing of transactions and lower customer satisfaction. With the RMS, TWG is now operating efficiently and effectively and is able to consistently deliver high levels of customer service.

RMS offers core point of sale functionality allowing you to process transactions as well as manage stock and inventory levels with facilities for online ordering and payment. RMS allows for real time synchronisation of information between stores and between warehouses. RMS provides a platform for both internal users (retail staff & management) and external users (online customers). All operation processes such as product scanning, processing a transaction at the POS, stock tracking & reporting are all handled by RMS.

The RMS also enables effective management by providing relevant information. With the RMS, managers are able to analyze sales information to forecast future trends, plan and manage inventory levels effectively, and review department performance.

The Retail Management System (RMS) is not just a transactional system for processing sales and services. It can also enable the efficient operation of the organisation as a whole, supporting analysis at all levels. The RMS brings together departmental data into a single source, reducing the scope for error caused by manual duplication and minimising the need for unnecessary data entry. It also facilitates better communication between departments, enabling

organisations to react more quickly to new opportunities, and better support their customers and clients.

In addition to processing a large number of in-store transactions the new RMS will also enable the Group to process online sales. The RMS must therefore be able to support the large number of stores for peak trade periods. “Our environment is busy, we need systems that can support us during peak periods, handling high volumes of transactions whilst still delivering a great in store experience to our customers. We need fast and accurate orders for our online customers, and real time analysis and insights for our management team”.

## Departmental structure and responsibilities:

The Warehouse Group is divided into a number of main departments which all have various interactions with the Retail Management System (RMS).

The **Retail Operations Department** is responsible for in-store activities, including customer service and point-of-sale transactions. Staff can scan items, make sales, figure out totals, and print receipts using the RMS at connected end points.

**Supply Chain and Logistics Department** distributes inventory, warehouse operations and restocking stock. This department uses the RMS to monitor stock levels, perform inventory updates and sends stock alerts

The **E-Commerce Departmen**t manages online sales platforms The RMS handles online tasks like product browsing, cart addition, checkout processing and delivery detail capturing.

The **Finance Department** takes care of processing payments, checking transactions, and making financial reports. The RMS works with payment systems to check and confirm transactions.

The **Information Technology** (IT) Department is in charge of making sure that the system infrastructure works and that the system is secure. They also handle connections to other systems, such as payment gateways.

The **Management Team** uses the RMS to make analytical reports like sales and inventory reports that help them make strategic decisions.

The interaction between departments is highly dependent on the RMS. For example, the Retail Operations Department relies on accurate inventory data from the Supply Chain Department, while the E-Commerce Department depends on real-time stock updates to ensure customers are not able to purchase unavailable products. This interdependency highlights the importance of having a well-integrated system.

## Identification of stakeholders and user roles:

Internal stakeholders are those who work within The Warehouse Group and directly engage with the RMS, while external stakeholders are individuals or groups who influence or are influenced by the operations of the system. Customers are the main external stakeholders, serving as users who directly engage with the online platform to search for products, add items to a shopping cart, and conduct transactions. There are internal users that include our Store associates, who will be interacting with the system while performing in-store transactions like scanning items, computing totals for customers, accepting payments and printing out orders. Warehouse Staff manages operations related to inventory, updating stock levels, checking product availability and responding to stock alerts from the system. Managers are also important stakeholders who use the RMS for analytical purposes as well as business performance monitoring along with strategic decision-makings. On the other hand, IT Administrators are also essential for system maintenance, security measures, user account management and enabling the integration of external services (ex: payment gateways). All these roles have different system interactions and permissions/functions granted also emphasizing the need of role-based access control in an RMS.

The RMS is attended by several parties with specific roles and responsibilities.

- Some of the primary stakeholders are **customers**, who use the system with online ordering processes where they browse products through an interface, add items to a cart and complete purchases.
- Upon acceptance, the **Sales Associates** are responsible for processing in-store transactions such as scanning products, computing totals (from introductory amounts to final price), settling payments and printing receipts.
- Maintaining, Updating Stocks, and Keeping track of stock levels using reporting tools, checking stock availability for order processing and handling stock discrepancy issues are some other inventory-related tasks managed by the **Warehouse Staff**.
- The system is used by **Managers** to generate reports, monitor performance and analyse sales and inventory data.
- **IT Administrators** keep systems running, manage user access, and ensure that the computer system is reliable and secure.

## System scope, boundaries and assumptions:

The Retail Management System caters for online and offline activities, order management and payment processing. It also manages stock (including POS scenarios) for advance payments for future deliveries. The system is omnichannel and enables in-store and online retailing capabilities.

The TWG operations systems that the team can control and have visibility into are included in the system scope for these operations, such as TWG stores, warehouses and online operations systems. However, third-party logistics systems are outside the scope of this Safety and Operations Plan. Integration points with third party applications can be managed as necessary.

This solution assumes that by providing a better way to report the Data of inventory and transaction, it will result in an increase to the system average time live. Role-based access control: The system will assign permissions to users based off of roles. High availability/ scalability: The platform will be designed to support large retail operations and be able to scale up during peak times. This means the system will be always available and running 24/7 with minimal to no downtime between operation. Accurate real-time data: The system will provide accurate, real-time information to support timely retail decisions. Access control: The system will control access to data and functions based off of the user's role to support system security and protect monetary transactions as well as sensitive customer information. Role-based access control protects your business, your customers and your data. Our solution helps the retailer manage and control access to information, safeguarding against potential security breaches.

# Task 2: Requirements Engineering

## Clearly defined functional requirements:

Functional requirements specify the behaviours that the Retail Management System needs to implement in order for business operations to continue.

**FR1**: The RMS should enable a sales associate to take the payment corresponding to a customer's purchase via a point-of-sale system.

**FR2**: The RMS should allow sales associates to scan products and fetch the details of products.

**FR3**: The RMS shall be able to compute total price for selected products automatically.

**FR4**: Receipt Generation: The RMS should be able to generate receipts for completed transactions.

**FR5**: The RMS shall update inventory levels in real time after each transaction.

**FR6:** The RMS shall enable customers to search the products via online.

**FR7**: The RMS shall permit customers to add the products in a shopping cart.

**FR8**: the RMS has to enable customers to follow a checkout process.

**FR9**: The RMS should record the customer's delivery details for its online orders.

**FR10**: The RMS shall process customer payments securely with integrated payment systems.

**FR11**: RMS should validate the payment information before processing any transactions.

**FR12**: The RMS shall verify successful payment transactions.

**FR13**: The RMS shall manage failed payment transactions and inform people when that happens.

**FR14**: Staff of the warehouse should have access to update stock level within the RMS.

**FR15**: The RMS shall allow users to check availability of stock

**FR16**: When the quantity of stock is below a defined level then RMS Should generate alerts.

**FR17**: The RMS shall allow managers to generate sales reports.

**FR18**: The RMS should enable managers to create stock reports.

**FR19**: RMS should provide a means for users to export reports.

**FR20**: the RMS shall provide an authenticated user access with role-based access control

## Clearly Defined non functional requirements:

**NFR1**: RMS shall respond to user actions within 2 seconds under normal operating conditions
.
**NFR2:** The RMS System should be reachable at all times with minimal downtime.

**NFR3**: The RMS shall provide secure processing of customer and payment data.

**NFR4**: The RMS should be able to scale for peak retail demand times.

**NFR5**: All users should be able to use an intuitive and user-friendly RMS.

**NFR6**: All stores and warehouses must have consistent data with the RMS.

**NFR7**: The RMS must operate in compliance with the applicable data protection and privacy regulations.

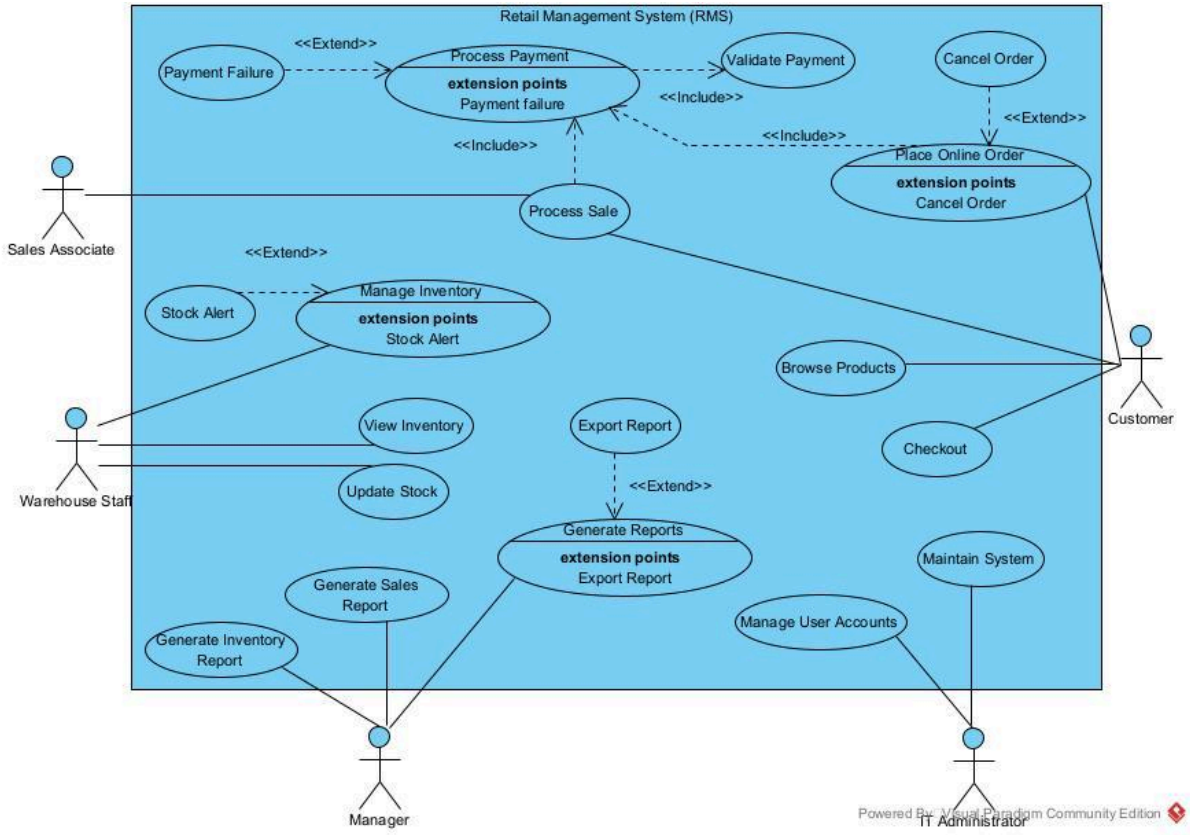
## Traceability between stakeholders and functional requirements:

| User Role | Related FRs | Related NFRs | Departments |
|---|---|---|---|
| Sales Associates | FR1, FR2, FR3, FR4, FR5, FR20 | NFR1, NFR5 | Retail Operations, Finance |
| Customers | FR6, FR7, FR8, FR9, FR 10, FR13 | NFR3 | E-Commerce |
| Warehouse Staff | FR14, FR15, FR16 | NFR2, NFR6 | Supply Chain/Logistics |
| Managers | FR17, FR18, FR19 | NFR1, NFR4 | Retail Operations, Supply Chain, Finance |
| IT Administrators | FR20 | NFR2, NFR3, NFR7 | IT (supports all departments) |

These are the requirements for the Use Case UML Diagram creation in Task 3.

# Task 3: UML Requirements Modelling

The Use Case Diagram below shows the overall functionality of Retail Management System (RMS) for all identified user roles and key departments across The Warehouse Group. Illustration of proper usage of include and extend relationships that properly represents system behaviours in both a mandatory and optional manner based on UML modelling principles. There is traceability of all use cases and relations with the functional requirements identified in Task 2, thus showing that there is consistency between system requirements and the Use Case Diagram




# Minimum of 5 Use Case Descriptions

| **ID** | **Use Case 1** |
|---|---|
| **Name** | Browse Products |
| **Description** | Customer browses products that are available in the system to view details and make the decision to purchase |
| **Pre-conditions** | System must be available |
| **Event Flow** | 1. Customer accesses the product catalogue<br>2. System displays available products<br>3. Customer filters or searches for products<br>4. System displays results matching to filter and/or searches<br>5. Customer views product details |
| **Post-conditions** | Customer has viewed product information and may proceed to purchase |
| **Extension points** | None |
| **Triggers** | Customer accesses the product catalogue |

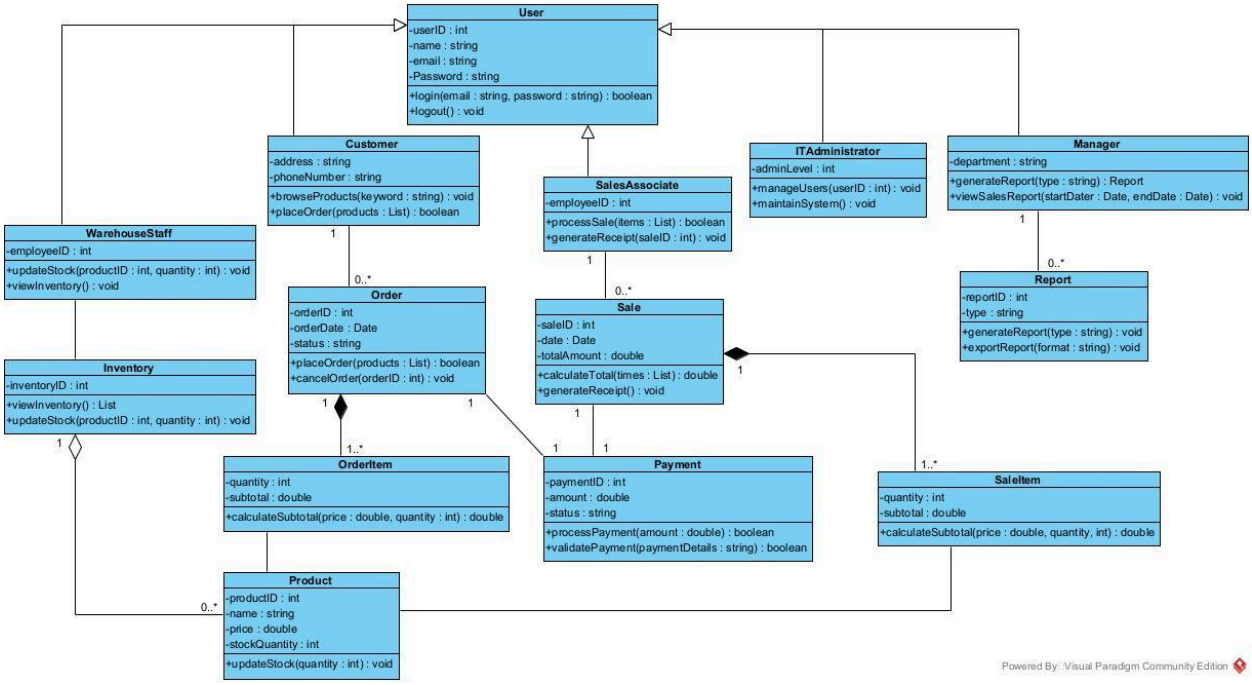
| ID | Use Case 2 |
|---|---|
| **Name** | Process Payment |
| **Description** | Handles validation and Processing of payments |
| **Pre-conditions** | Transaction initiated and payment details provided |
| **Event Flow** | 1. System receives payment details<br>2. Include “Validate Payment”<br>3. System confirms payment completion |
| **Post-conditions** | Payment approved or declined |
| **Extensions** | Payment Failure extends Process Payment - if validation fails |
| **Triggers** | Payment details are submitted |

| ID | Use Case 3 |
|---|---|
| **Name** | Place Online Order |
| **Description** | Customer places an order through the online system |
| **Pre-condition** | Customer has selected products |
| **Event flow** | 1. Customer selects products<br>2. Customer proceeds to checkout<br>3. System displays order summary<br>4. Include “Process Payment”<br>5. System confirms order |
| **Post-condition** | Order is placed and payment is processed |
| **Extension points** | Cancel Order extends Place Online Order-if the customer cancels before confirmation |
| **Triggers** | Customer proceeds to checkout |

| **ID** | **Use Case 4** |
|---|---|
| **Name** | Manage Inventory |
| **Description** | Warehouse Staff manages stock levels and updates inventory |
| **Pre-condition** | Warehouse staff must be authorised |
| **Event flow** | 1. Staff accesses inventory system<br>2. System displays inventory details<br>3. Staff updates stock levels<br>4. System saves updated inventory |
| **Post-condition** | Inventory records are updated |
| **Extension points** | Stock Alert extends Manage Inventory - if stock levels fall below threshold |
| **Triggers** | Warehouse Staff accesses inventory system |

| **ID** | **Use Case 5** |
|---|---|
| **Name** | Generate reports |
| **Description** | Manager generates reports based on system data |
| **Pre-condition** | Manager is authorised |
| **Event flow** | 1. Manager requests report<br>2. System retrieves relevant data<br>3. System generates report<br>4. System displays report |
| **Post-condition** | Report is generated and is available for viewing |
| **Extension points** | Export Report extends Generate Reports - if the manager chooses export |
| **Triggers** | Manager requests a report |

# Task 4: Initial Structural design and Implementation Planning




## Mapping between UML classes and planned C# classes:

During the analysis phase of Retail Management System (RMS) design, a part of its structure has been specified as a UML class diagram. This diagram can be used to generate an object-oriented implementation of RMS in C#. In UML classes are mapped to C# classes, classes' attributes in UML are mapped to C# class fields, and classes' operations in UML are mapped to C# methods. A class User has all common attributes of all users of RMS: UserID, name, email, and password. There are two methods in User class: Login() and Logout(). Specialized versions of User class – classes of Customer, SalesAssociate, WarehouseStaff, Manager, and ITAdministrator – also have all the properties and methods of User class and their own attributes and methods. The example below demonstrates how the Customer class has extra attributes like address and phoneNumber as well as methods like BrowseProducts() and PlaceOrder(). Similarly the Manager class has methods to make reports and view reports. Some of the main business entities like Product, Order, Sale, Payment, Inventory and Report are directly mapped to corresponding C# classes. Looking at the UML diagram we can see the attributes like productID, price and stockQuantity as variables in the product class and operations like UpdateStock() as methods. two new classes were added: Order and Sale. They have fields to keep the transaction information. Methods were implemented to process the transaction and add items to the transaction (OrderItem to Order and SaleItem to Sale). The composition relationships between the two new classes and their items are represented in the UML diagram. These relationships are represented in the C# implementation using composition

relationships where one class contains a list of items (Order and Sale) and object references to implement the association relationships between Customer and Order, and Manager and Report. The mapping between the given UML design and the C# structure is clear and therefore supports architecture consistency, modularity and object-oriented structure. Besides the structural relationships between the entities, the class diagram adheres to further object-oriented design principles. The principles of encapsulation, inheritance and modularity have been considered when designing the classes in the class diagram. Encapsulation is the association of data with the methods that manipulate it in order to ensure data safety. In the class diagram the Product class encapsulates the attributes price and stock quantity and provides a method to safely change the stock quantity. Inheritance is used correctly in your User class hierarchy to enable reuse of attributes and methods among different roles. By reusing inheritance, you get increased modularity, less repeated code, and so on. Additionally, you have separated out components of the system such as Order, Payment, and and existing code for an Inventory. This will help to organize your system and make it easier to expand/maintain in the future.

## Identification of anticipated design weaknesses prior to OOP application:

Using object-oriented programming (OOP) in our Retail Management System helped to prevent many design flaws that would have made the system hard to maintain, expand, and keep all components coherent. High coupling would have been a huge problem. Processes like processing a sale, tracking inventory, and payment processing are all related. Without OOP, as one process changed, many other processes would be negatively affected. Low cohesion would have also been a huge problem. Functions like processing a sale, tracking inventory, and generating reports would all be contained in a single module instead of each having their own separate module with single responsibility. This would make for hard to read, hard to understand, and hard to maintain code. Duplicate code. Because the same logic is reused in the design, for example to validate a payment or manipulate a product, this will lead to slower development because it is hard to maintain all the duplicate code at the same time. It will also be hard to have encapsulation within the system, which means the data like product details, customer details and transaction history can be easily exposed, which will lead to the increased chances of data corruption and overall system security breach. The system cannot be reused/extended to fit another business. Adding in new features, for example adding in more user roles or payment options, would make the system much harder to test and maintain because it lacks modularity. Therefore, application of key OOP concepts were not used in the design of the Retail Management System. As the project was initially conceived using object-oriented techniques, it was felt that the system could become unstable as the number of users rose. Without a well designed system architecture and module structure the “end of season” retail periods where all customers are processing at the same time could have led to problems with system performance. The code does not have sufficient separation of concerns between the system's components. This makes the code harder to test and to debug. Because

there is no separation of concerns, errors in one piece of code could potentially affect other parts of the code. This makes development take more time and results in code of lower quality. Without abstraction, the script would be very rigid and inflexible and would become very painful to add in support for new payment methods or for additional reporting features. Using object-oriented programming practices allows us to build a flexible, maintainable, and extensible script. Interfaces and Enumerations are not shown in the above design diagram since they are not required to represent class structure and their interconnections. However, these can enhance the code structure in future. For example, an OrderStatusEnum can be created to define the different statuses allowed for an Order and these statuses can be ordered based on precedence. Similarly, instead of inheriting from different classes for different payment mechanisms, an interface IPayable can be defined for both Order and Invoice classes to generalize the payment logic. These enhancements can be made later during the actual design and implementation of the solution using C#.